

ใบงานที่ 1

เรื่อง การทดลองวงจรมัลติเพลกเซอร์และวงจรมัลติเพลกเซอร์

(Multiplexer and Demultiplexer)

จัดทำโดย

นาย ศิริพงศ์ บุญประสิทธิ์

รหัส ENGCE200_SEC1

เสนอ

อาจารย์สมนึก สุระธง

ใบงานนี้เป็นส่วนหนึ่งของวิชา

ENGCE200

การออกแบบดิจิทัล

(Digital Systems Design)

หลักสูตร วศ.บ.วิศวกรรมคอมพิวเตอร์

สาขาวิศวกรรมไฟฟ้า คณะวิศวกรรมศาสตร์

มหาวิทยาลัยเทคโนโลยีราชมงคลล้านนา ภาคพายัพ เชียงใหม่

ภาคเรียนที่ 1 ปีการศึกษา 2568

ใบงานที่ 1

ชื่อบทเรียน การทดลองวงจรมัลติเพลกเซอร์และวงจรดีมัลติเพลกเซอร์

(Multiplexer and Demultiplexer)

จุดประสงค์การสอน

ปฏิบัติการทดลองสร้างวงจรคอมไบเนชันด้วยลอจิกเกตพื้นฐาน

ออกแบบและทดสอบวงจรมัลติเพล็กซ์

ออกแบบและทดสอบวงจรดีมัลติเพล็กซ์

1. ทฤษฎี

วิธีการออกแบบวงจรมัลติเพล็กซ์ (Multiplexer)

วิธีการออกแบบวงจรดีมัลติเพล็กซ์ (Demultiplexer)

2. จงออกแบบวงจรต่อไปนี้

1. วงจรมัลติเพล็กซ์ (16 : 1 Multiplexer)

2. วงจรดีมัลติเพล็กซ์ (1 : 16 Demultiplexer)

3. ผลการทดลอง

ทดลองโดยการนำ Output Multiplexer ต่อ เข้า Input Demultiplexer แล้วเลือก select อิสระ

4. สรุปผลการทดลอง

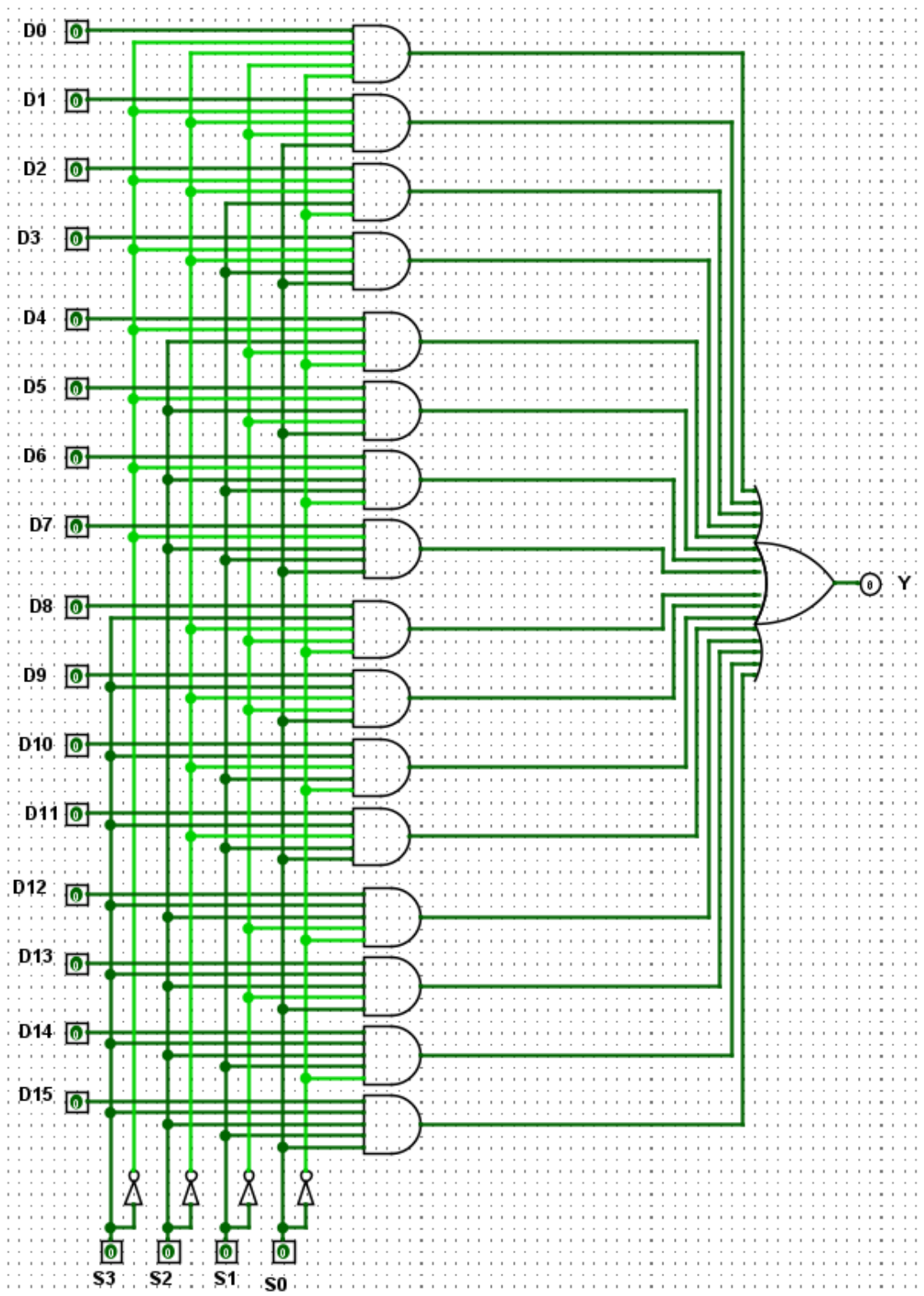
5. คำถามหลังการทดลอง

1. บอกหลักการทำงานของวงจรมัลติเพล็กซ์

2. บอกหลักการทำงานของวงจรดีมัลติเพล็กซ์

3. บอกวิธีการเชื่อมต่อทางไกลโดยสายส่งคู่เดียว

Multiplexer 16:1 logic



MUX16to1.vhdl

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity MUX16to1 is
    port (
        D : in std_logic_vector(15 downto 0);
        S : in std_logic_vector(3 downto 0);
        Y : out std_logic
    );
end MUX16to1;

architecture GateLevel of MUX16to1 is

    signal W : std_logic_vector(15 downto 0);

begin

    W(0) <= D(0) and (not S(3)) and (not S(2)) and (not S(1)) and (not S(0));
    W(1) <= D(1) and (not S(3)) and (not S(2)) and (not S(1)) and S(0);
    W(2) <= D(2) and (not S(3)) and (not S(2)) and S(1) and (not S(0));
    W(3) <= D(3) and (not S(3)) and (not S(2)) and S(1) and S(0);

    W(4) <= D(4) and (not S(3)) and S(2) and (not S(1)) and (not S(0));
    W(5) <= D(5) and (not S(3)) and S(2) and (not S(1)) and S(0);
    W(6) <= D(6) and (not S(3)) and S(2) and S(1) and (not S(0));
    W(7) <= D(7) and (not S(3)) and S(2) and S(1) and S(0);

    W(8) <= D(8) and S(3) and (not S(2)) and (not S(1)) and (not S(0));
    W(9) <= D(9) and S(3) and (not S(2)) and (not S(1)) and S(0);
```

```
W(10) <= D(10) and S(3) and (not S(2)) and S(1) and (not S(0));
W(11) <= D(11) and S(3) and (not S(2)) and S(1) and S(0);

W(12) <= D(12) and S(3) and S(2) and (not S(1)) and (not S(0));
W(13) <= D(13) and S(3) and S(2) and (not S(1)) and S(0);
W(14) <= D(14) and S(3) and S(2) and S(1) and (not S(0));
W(15) <= D(15) and S(3) and S(2) and S(1) and S(0);

Y <= W(0) or W(1) or W(2) or W(3) or
    W(4) or W(5) or W(6) or W(7) or
    W(8) or W(9) or W(10) or W(11) or
    W(12) or W(13) or W(14) or W(15);

end GateLevel;
```

tb_MUX16to1.vhdl

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity tb_MUX16to1 is
end tb_MUX16to1;

architecture sim of tb_MUX16to1 is

    signal D0 : std_logic := '0';
    signal D1 : std_logic := '0';
    signal D2 : std_logic := '0';
    signal D3 : std_logic := '0';
    signal D4 : std_logic := '0';
    signal D5 : std_logic := '0';
    signal D6 : std_logic := '0';
    signal D7 : std_logic := '0';
    signal D8 : std_logic := '0';
    signal D9 : std_logic := '0';
    signal D10 : std_logic := '0';
    signal D11 : std_logic := '0';
    signal D12 : std_logic := '0';
    signal D13 : std_logic := '0';
    signal D14 : std_logic := '0';
    signal D15 : std_logic := '0';

    signal D : std_logic_vector(15 downto 0);

    signal S : std_logic_vector(3 downto 0);
    signal Y : std_logic;

begin
```

```
D <= D15 & D14 & D13 & D12 &  
D11 & D10 & D9 & D8 &  
D7 & D6 & D5 & D4 &  
D3 & D2 & D1 & D0;
```

```
uut : entity work.MUX16to1  
  port map  
  (  
    D => D,  
    S => S,  
    Y => Y  
  );
```

```
process  
begin
```

```
D0 <= '1';  
S  <= "0000";  
wait for 10 ns;  
D0 <= '0';
```

```
D1 <= '1';  
S  <= "0001";  
wait for 10 ns;  
D1 <= '0';
```

```
D2 <= '1';  
S  <= "0010";  
wait for 10 ns;  
D2 <= '0';
```

```
D3 <= '1';  
S  <= "0011";  
wait for 10 ns;  
D3 <= '0';
```

```
D4 <= '1';  
S  <= "0100";  
wait for 10 ns;  
D4 <= '0';
```

```
D5 <= '1';  
S  <= "0101";  
wait for 10 ns;  
D5 <= '0';
```

```
D6 <= '1';  
S  <= "0110";  
wait for 10 ns;  
D6 <= '0';
```

```
D7 <= '1';  
S  <= "0111";  
wait for 10 ns;  
D7 <= '0';
```

```
D8 <= '1';  
S  <= "1000";  
wait for 10 ns;  
D8 <= '0';
```

```
D9 <= '1';  
S  <= "1001";  
wait for 10 ns;
```



```
D9 <= '0';

D10 <= '1';
S   <= "1010";
wait for 10 ns;
D10 <= '0';

D11 <= '1';
S   <= "1011";
wait for 10 ns;
D11 <= '0';

D12 <= '1';
S   <= "1100";
wait for 10 ns;
D12 <= '0';

D13 <= '1';
S   <= "1101";
wait for 10 ns;
D13 <= '0';

D14 <= '1';
S   <= "1110";
wait for 10 ns;
D14 <= '0';

D15 <= '1';
S   <= "1111";
wait for 10 ns;
D15 <= '0';
```

```

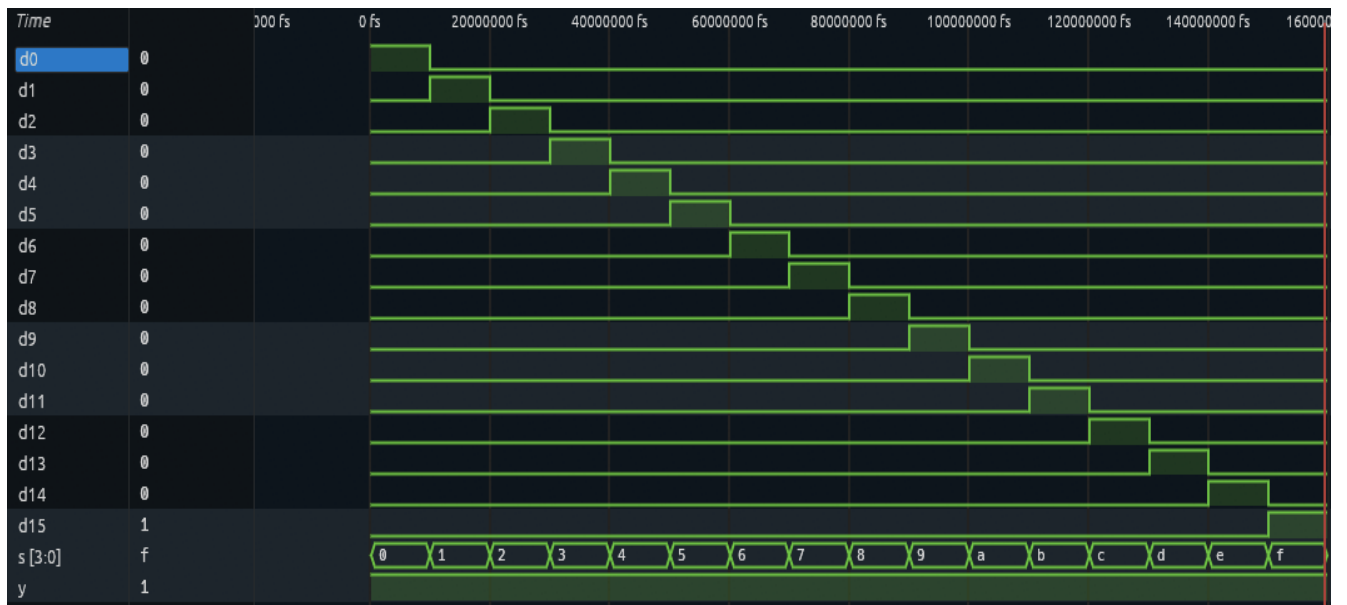
    wait;

    end process;

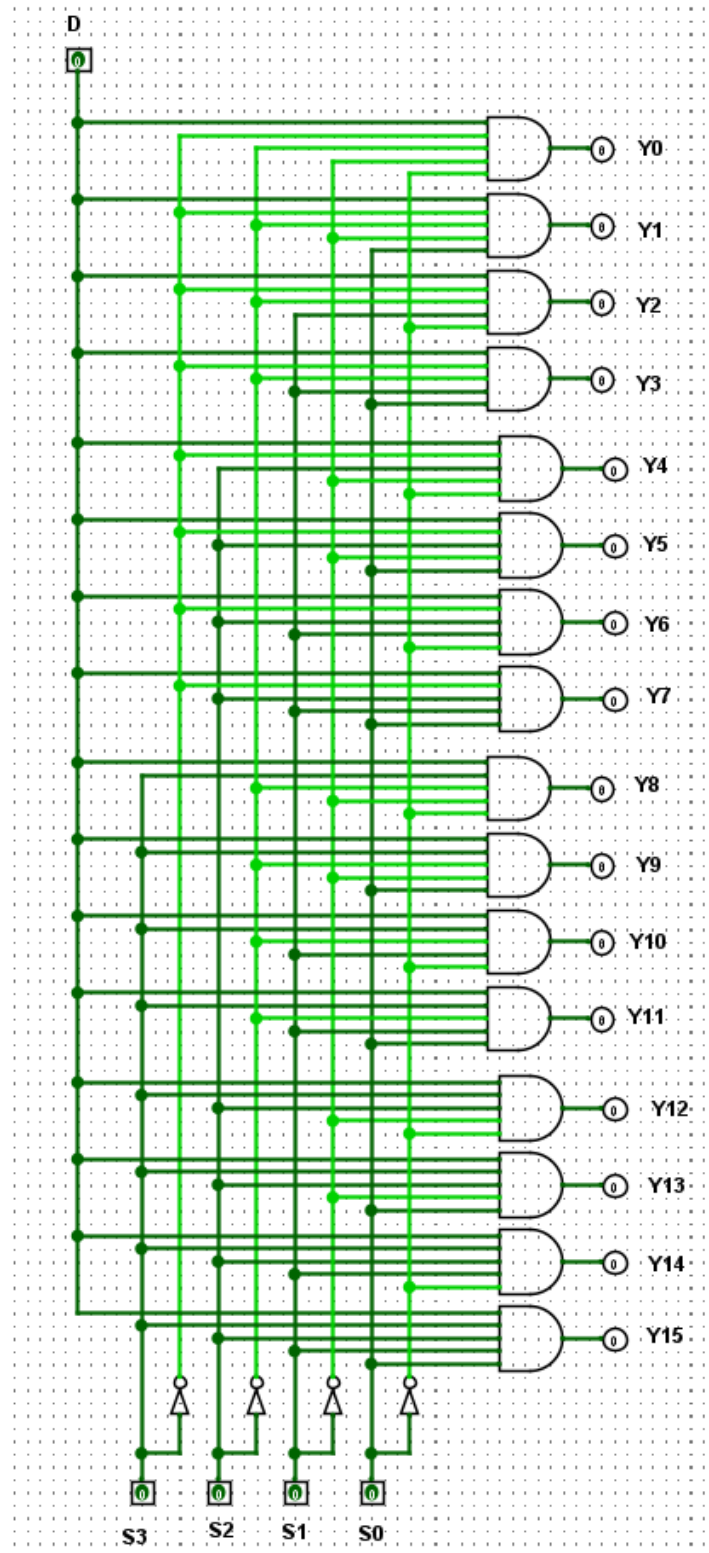
end sim;

```

MUX16-1-wave.vcd



Demultiplexer 1:16 logic



DMUX1to16.vhdl

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity DMUX1to16 is
    port (
        D : in std_logic;
        S : in std_logic_vector(3 downto 0);
        Y : out std_logic_vector(15 downto 0)
    );
end DMUX1to16;

architecture GateLevel of DMUX1to16 is
begin

    Y(0) <= D and (not S(3)) and (not S(2)) and (not S(1)) and (not
S(0)); -- 0000
    Y(1) <= D and (not S(3)) and (not S(2)) and (not S(1)) and S(0);
    Y(2) <= D and (not S(3)) and (not S(2)) and S(1) and (not S(0));
    Y(3) <= D and (not S(3)) and (not S(2)) and S(1) and S(0);

    Y(4) <= D and (not S(3)) and S(2) and (not S(1)) and (not S(0));
    Y(5) <= D and (not S(3)) and S(2) and (not S(1)) and S(0);
    Y(6) <= D and (not S(3)) and S(2) and S(1) and (not S(0));
    Y(7) <= D and (not S(3)) and S(2) and S(1) and S(0);

    Y(8) <= D and S(3) and (not S(2)) and (not S(1)) and (not S(0));
    Y(9) <= D and S(3) and (not S(2)) and (not S(1)) and S(0);
    Y(10) <= D and S(3) and (not S(2)) and S(1) and (not S(0));
    Y(11) <= D and S(3) and (not S(2)) and S(1) and S(0);

    Y(12) <= D and S(3) and S(2) and (not S(1)) and (not S(0));
```

```
Y(13) <= D and S(3) and S(2) and (not S(1)) and S(0);  
Y(14) <= D and S(3) and S(2) and S(1) and (not S(0));  
Y(15) <= D and S(3) and S(2) and S(1) and S(0); -- 1111  
  
end GateLevel;
```

tb_DMUX1to16.vhdl

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity tb_DMUX1to16 is
end tb_DMUX1to16;

architecture sim of tb_DMUX1to16 is

    signal D : std_logic := '0';
    signal S : std_logic_vector(3 downto 0);

    signal Y : std_logic_vector(15 downto 0);

    signal Y0 : std_logic;
    signal Y1 : std_logic;
    signal Y2 : std_logic;
    signal Y3 : std_logic;
    signal Y4 : std_logic;
    signal Y5 : std_logic;
    signal Y6 : std_logic;
    signal Y7 : std_logic;
    signal Y8 : std_logic;
    signal Y9 : std_logic;
    signal Y10 : std_logic;
    signal Y11 : std_logic;
    signal Y12 : std_logic;
    signal Y13 : std_logic;
    signal Y14 : std_logic;
    signal Y15 : std_logic;
```

```
begin

    uut : entity work.DMUX1to16
        port map
        (
            D => D,
            S => S,
            Y => Y
        );

    Y0 <= Y(0);
    Y1 <= Y(1);
    Y2 <= Y(2);
    Y3 <= Y(3);
    Y4 <= Y(4);
    Y5 <= Y(5);
    Y6 <= Y(6);
    Y7 <= Y(7);
    Y8 <= Y(8);
    Y9 <= Y(9);
    Y10 <= Y(10);
    Y11 <= Y(11);
    Y12 <= Y(12);
    Y13 <= Y(13);
    Y14 <= Y(14);
    Y15 <= Y(15);

    process
    begin

        D <= '1';

        S <= "0000";
```

```
wait for 10 ns;  
S <= "0001";  
wait for 10 ns;  
S <= "0010";  
wait for 10 ns;  
S <= "0011";  
wait for 10 ns;
```

```
S <= "0100";  
wait for 10 ns;  
S <= "0101";  
wait for 10 ns;  
S <= "0110";  
wait for 10 ns;  
S <= "0111";  
wait for 10 ns;
```

```
S <= "1000";  
wait for 10 ns;  
S <= "1001";  
wait for 10 ns;  
S <= "1010";  
wait for 10 ns;  
S <= "1011";  
wait for 10 ns;
```

```
S <= "1100";  
wait for 10 ns;  
S <= "1101";  
wait for 10 ns;  
S <= "1110";  
wait for 10 ns;  
S <= "1111";
```



```
wait for 10 ns;

D <= '0';

wait;

end process;

end sim;
```

DMUX1-16-wave.vcd

